Unsupervised Learning: Clustering and Dimensionality Reduction

Course Reading Material

CIS 635: Knowledge Discovery and Data Mining
Grand Valley State University

Fall 2026

© 2026 Kamrul Hasan. Prepared for students of CIS 635. Questions and corrections: hasaka@gvsu.edu.
Please cite Hasan (2026) if you reuse this material.

Unit: Unsupervised Learning — Clustering and Dimensionality Reduction

This reading accompanies the lecture slides *Introduction to Unsupervised Learning*, *Clustering Algorithms*, *Selecting the Number of Clusters*, and *Linear Dimensionality Reduction (PCA)*. It is a companion to the Regression and Classification readings: those readings taught you to learn a mapping from features to a known label; this one asks what we can learn when there is no label at all. It also builds on the distance measures from the Advanced EDA reading, the scaling material from the Feature Scaling notebook, and the curse-of-dimensionality discussion in the Feature Selection reading. A companion Python script, `unsupervised-learning_example.py`, reproduces every number reported in Sections 13 and 14.

Contents

Notation and Terminology Quick Reference **2**

Learning Objectives **3**

1 Unsupervised Learning in the KDD Process **3**

2 Similarity, Scaling, and Mixed Data **4**

3 What Is Clustering? **5**

4 k-Means and k-Modes **6**

5 Hierarchical Clustering **8**

6 Gaussian Mixture Models **9**

7 DBSCAN: Density-Based Clustering **12**

8 How Many Clusters? **13**

9 Evaluating and Interpreting Clusters **15**

10 The Curse of Dimensionality **15**

11 Principal Component Analysis **16**

12 Nonlinear Embeddings for Visualization: t-SNE and UMAP **19**

13 Python Implementation **20**

14 Case Study: Do Natural Groups Match the Diagnosis? **21**

15 Common Pitfalls **22**

16 Key Takeaways **23**

Wrap-Up: Connecting Back **23**

Review Questions **24**

Suggested Hands-on Exercise **24**

Notation and Terminology Quick Reference

Symbol / term	Meaning	§
$X \in \mathbb{R}^{n \times p}$	Data matrix: n observations (rows), p features (columns); no label y	1
K (or k)	Number of clusters, or number of mixture components	4
C_j, $\boldsymbol{\mu}_j$	Cluster j and its centroid (mean vector)	4
WCSS (inertia)	Within-cluster sum of squares, $\sum_j \sum_{\mathbf{x} \in C_j} \lVert \mathbf{x} - \boldsymbol{\mu}_j \rVert^2$	4
Mode	Most frequent value of a categorical variable	4
Linkage	Rule for the distance between two *groups* of points (single, complete, average, Ward)	5
Dendrogram	Tree that records the order and height of hierarchical merges	5
π_k	Mixing weight of GMM component k; $\pi_k \geq 0$, $\sum_k \pi_k = 1$	6
$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$	Gaussian density with mean $\boldsymbol{\mu}_k$ and covariance $\boldsymbol{\Sigma}_k$	6
γ_{ik} (responsibility)	Soft assignment $p(z_k = 1 \mid \mathbf{x}_i)$: probability point i came from component k	6
EM	Expectation–Maximization: alternate soft assignment (E) and parameter update (M)	6
ε, minPts	DBSCAN neighbourhood radius and density threshold	7
Core / border / noise	DBSCAN point types	7
TV, BCV	Total and between-cluster variation (sums of squares); TV = BCV + WCSS	8
$s(i)$, silhouette	$s(i) = (b_i - a_i)/\max(a_i, b_i) \in [-1, 1]$; the silhouette score is the mean of $s(i)$	8
BIC	Bayesian information criterion; lower is better	8
ARI	Adjusted Rand index: agreement between two partitions, 0 = chance, 1 = identical	9
$\mathbf{S}$ (or A)	Sample covariance matrix, $p \times p$ and symmetric	11
λ_j, $\mathbf{u}_j$	j-th eigenvalue and eigenvector of $\mathbf{S}$; $\mathbf{u}_j$ is the j-th principal component	11
Loading	Entry of $\mathbf{u}_j$: weight of an original feature in a component	11
Explained variance ratio	$\lambda_j / \sum_l \lambda_l$	11
t-SNE, UMAP	Nonlinear neighbour-preserving embeddings, mainly for visualization	12

Learning Objectives

After working through this reading you should be able to:

1. Distinguish supervised learning, which models $f(y \mid X)$, from unsupervised learning, which models $f(X)$, and name clustering and dimensionality reduction as its two main tasks.
2. Explain why feature scaling and the choice of distance decide what a clustering can find, and handle categorical and mixed data (k-modes, Gower distance, k-prototypes).
3. Execute k-means and k-modes by hand on a small example, and describe k-means++ initialization and the role of multiple restarts.
4. Describe agglomerative and divisive hierarchical clustering, read a dendrogram, and compare linkage rules.
5. State the Gaussian mixture model, compute soft assignments with Bayes' rule, and outline the EM algorithm.
6. Explain DBSCAN's core, border, and noise points, and choose ε with a k-distance plot.
7. Choose the number of clusters with the elbow (explained variance), silhouette, and BIC criteria, and evaluate clusters internally and externally (ARI).
8. Explain the curse of dimensionality, derive PCA from the covariance matrix, and decide how many components to keep.
9. Use t-SNE and UMAP responsibly for visualization, knowing what their plots can and cannot tell you.

1 Unsupervised Learning in the KDD Process

In the Regression and Classification readings every row of the data came with an answer. A patient's record carried a diabetes label; a house carried a sale price. The learning task was to find the relationship between the features X and the label y, written $f(y \mid X)$, for example $f(y = \text{diabetes} \mid X = [\text{age}, \text{bmi}])$. That is **supervised** (or discriminative) learning. When y is a real number ($y \in \mathbb{R}$: insurance cost, house price, energy use) the task is regression; when y comes from a closed set ($y \in \{\text{spam}, \text{not spam}\}$, $y \in \{0, 1, \ldots, 9\}$) it is classification.

Unsupervised learning drops the label. We have only the feature matrix X, and the goal is to learn a meaningful representation or model of X itself, written $f(X)$. There is no column to predict and therefore no accuracy to compute.

Table 1: Supervised versus unsupervised learning.

	Supervised	**Unsupervised**
Data	Features X and a predefined label y	Features X only
What is learned	$f(y \mid X)$: a mapping to the label	$f(X)$: structure in X (groups, directions, density)
Typical tasks	Regression, classification	Clustering, dimensionality reduction, density estimation, anomaly detection
Output for a new row	A predicted value or class	A group index, or new (fewer) coordinates
How we judge success	Error on held-out labels	Internal criteria, stability, and whether a domain expert finds the result useful

Consider a medical study that records, for each patient, the *average number of symptoms per week* and their *average severity*. Plotted, the points form three visible clouds even though nobody labelled the patients. A clustering algorithm turns those clouds into three group indices; a clinician might then recognize them as, say, "few but severe symptoms," "many mild symptoms," and "many severe symptoms." The algorithm found the groups; the human named them.

This reading covers the two workhorses of unsupervised learning:

- **Clustering** (Sections 3–9) groups the rows of X so that rows in the same group are similar and rows in different groups are not.
- **Dimensionality reduction** (Sections 10–12) compresses the columns of X from p features to $d < p$ new features while keeping as much of the structure as possible.

Good general treatments of the material in this reading are Han et al. (2023, ch. 10–11), James et al. (2023, ch. 12), and Hastie et al. (2009, ch. 14). In the KDD process, unsupervised methods appear in two places. During *exploration* they reveal structure you did not know to look for (customer segments, disease subtypes, sensor sites that behave alike). During *preparation* they produce inputs for later modeling: PCA components can replace hundreds of correlated features, and a cluster index can become a new feature for a classifier.

Key Idea

Without a label there is no single "right answer" to score against. A clustering is a *hypothesis* about structure in the data. It is only as good as (1) the features and distance you chose, (2) the assumptions built into the algorithm, and (3) the evidence that the groups are stable and meaningful to someone who knows the domain.

2 Similarity, Scaling, and Mixed Data

Every clustering algorithm in this reading groups points that are "close." Closeness is measured with a distance, so the distance decides what the algorithm can find. The Advanced EDA reading introduced Euclidean (L_2), Manhattan (L_1), and cosine distances; here we focus on two practical consequences.

2.1 Scale first

Euclidean distance adds squared differences across features. A feature measured in large units therefore dominates. The Wine data set (178 wines, 13 chemical measurements, 3 grape cultivars) is a clean example: `proline` ranges over 1,402 units while `hue` ranges over about 1.2. Running k-means with $K = 3$ on the raw features and then comparing the clusters with the (hidden) cultivar gives an adjusted Rand index (ARI, Section 9) of only **0.371**. After standardizing every feature to mean 0 and standard deviation 1, the same algorithm reaches **0.897**. Nothing changed except the units.

Common Pitfall

Clustering raw features silently means "cluster by whichever feature has the biggest numbers." Standardize (or otherwise put features on comparable scales) before any distance-based method: k-means, hierarchical clustering, DBSCAN, and PCA. The exception is when the original units are meaningful and comparable, such as pixel intensities or coordinates in the same unit.

2.2 Categorical and mixed data

The mean of "red" and "green" does not exist, so k-means only works for numeric data. Three common responses:

1. **Convert everything to one type.** One-hot encode categories and treat the 0/1 columns as numbers, or bin numeric features into categories. Both are easy but have costs: one-hot columns inflate dimensionality and their scale must be balanced against the numeric columns; binning throws away within-bin detail and depends on where you draw the bin edges.
2. **Use a categorical algorithm.** *k-modes* (Huang, 1998) replaces means with modes and Euclidean distance with a simple mismatch count (Section 4.4).
3. **Use a mixed-type distance or algorithm.** *Gower's distance* (Gower, 1971) averages a per-feature dissimilarity in $[0, 1]$: a range-scaled absolute difference for numeric features and 0/1 mismatch for categorical ones. *k-prototypes* (Huang, 1998) combines k-means on the numeric part with k-modes on the categorical part, so it updates both a mean and a mode for each centroid.

Worked Example: Gower Distance Between Two Patients

Patient A: age 45, BMI 24, smoker = yes, blood pressure = high. Patient B: age 60, BMI 30, smoker = no, blood pressure = high. Suppose age ranges over 20–80 (range 60) and BMI over 15–45 (range 30) in the data.

$$d_{\text{age}} = \frac{|45-60|}{60} = 0.25, \quad d_{\text{BMI}} = \frac{|24-30|}{30} = 0.20, \quad d_{\text{smoker}} = 1, \quad d_{\text{BP}} = 0.$$

Gower distance $= (0.25 + 0.20 + 1 + 0)/4 = 0.3625$. Each feature contributes at most 1, so no feature can dominate because of its units. Gower distances can be fed to hierarchical clustering or to k-medoids (Kaufman and Rousseeuw, 1990), which only need pairwise distances.

3 What Is Clustering?

Clustering groups data points using their features X; the output is a **group index** for each point. Imagine six apples: small, medium, and large, each in red or green. Group them by *color* and you get two clusters; by *size*, three clusters; by both size and color, six. A new apple is then "labelled" with the index of the group it falls in. The lesson is that the features you choose define what "similar" means, and different reasonable choices give different reasonable clusterings.

Clustering algorithms differ mainly in what they think a cluster *is*. Table 2 lists the four families covered in the slides.

Table 2: Four families of clustering algorithms.

Family	Example	A cluster is…	Typical weakness
Centroid-based	k-means, k-modes	Points closest to a centre	Assumes round, similar-sized groups; needs K
Connectivity-based	Hierarchical (agglomerative)	Points linked by short distances	$O(n^2)$ memory; linkage choice matters
Distribution-based	Gaussian mixture model	Points generated by one probability distribution	Assumes a distribution shape; needs K
Density-based	DBSCAN	A contiguous region of high point density	Struggles with varying density and many dimensions

Clustering is used for market segmentation (customers grouped for marketing), social network analysis (communities of densely connected people), search-result grouping, medical imaging and image segmentation (pixels grouped into tissue types or objects), and anomaly detection (points that belong to no group, or to a tiny one).

4 k-Means and k-Modes

4.1 The k-means algorithm

k-means (Lloyd, 1982; MacQueen, 1967) is centroid-based, works only on numeric data, needs K as input, and is iterative. Given K initial means $\boldsymbol{\mu}_1^{(1)}, \ldots, \boldsymbol{\mu}_K^{(1)}$ it alternates two steps:

1. **Assignment step.** Assign each point to the nearest mean (least squared Euclidean distance):

$$C_j^{(t)} = \left\{ \mathbf{x}_i : \|\mathbf{x}_i - \boldsymbol{\mu}_j^{(t)}\|^2 \le \|\mathbf{x}_i - \boldsymbol{\mu}_l^{(t)}\|^2 \ \ \forall l \right\}.$$

2. **Update step.** Recompute each mean as the average of its points: $\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{|C_j^{(t)}|} \sum_{\mathbf{x}_i \in C_j^{(t)}} \mathbf{x}_i$.

   In two dimensions this is $m_x = \frac{1}{|C_j|} \sum x_i$ and $m_y = \frac{1}{|C_j|} \sum y_i$ over the cluster's points.

Stop when the means no longer move (equivalently, no point changes cluster) or after a set number of iterations. Each step can only lower the **within-cluster sum of squares**,

$$\text{WCSS} = \sum_{j=1}^{K} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2,$$

which scikit-learn calls `inertia_`. Because WCSS never increases and there are finitely many partitions, k-means always converges, but only to a *local* minimum.

Worked Example: k-Means by Hand ($K = 2$, Five Points)

Points: $A(1,1)$, $B(2,1)$, $C(4,3)$, $D(5,4)$, $E(1.5,2)$. Start with the means at A and B: $\boldsymbol{\mu}_1 = (1,1)$, $\boldsymbol{\mu}_2 = (2,1)$.

Iteration 1. Distances to $(\boldsymbol{\mu}_1, \boldsymbol{\mu}_2)$: $A\,(0,\ 1)$, $B\,(1,\ 0)$, $C\,(3.61,\ 2.83)$, $D\,(5.00,\ 4.24)$, $E\,(1.12,\ 1.12)$. E is a tie; we keep it with cluster 1 (lowest index), so $C_1 = \{A, E\}$ and $C_2 = \{B, C, D\}$. New means: $\boldsymbol{\mu}_1 = (1.25, 1.5)$, $\boldsymbol{\mu}_2 = (3.67, 2.67)$; WCSS $= 9.96$.

Iteration 2. B is now closer to $\boldsymbol{\mu}_1$ (0.90 versus 2.36), so $C_1 = \{A, B, E\}$ and $C_2 = \{C, D\}$.

New means: $\boldsymbol{\mu}_1 = (1.5, 1.33)$, $\boldsymbol{\mu}_2 = (4.5, 3.5)$; WCSS $= 2.17$.

Iteration 3. No point changes cluster, so the means stop moving. Done.

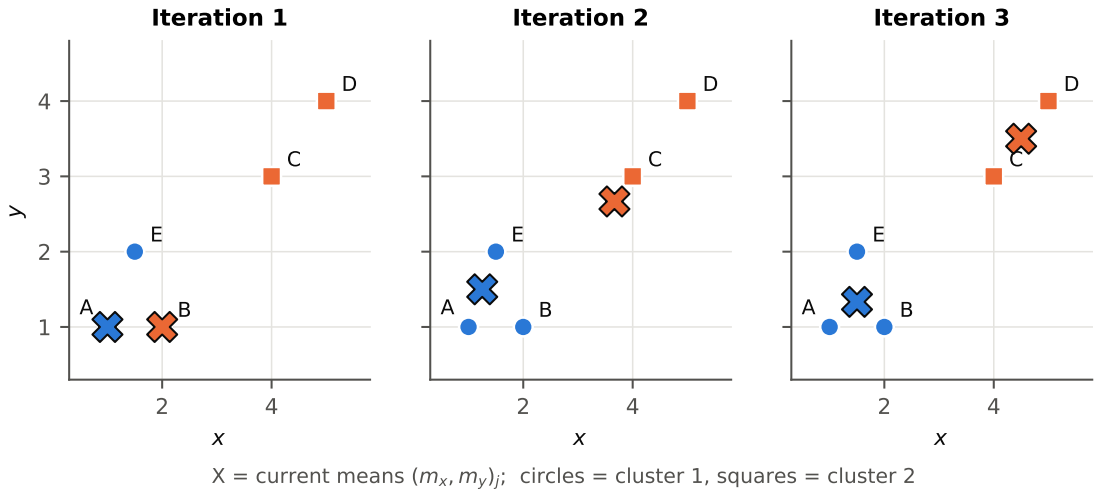


Figure 1: The worked example. Iteration 1 starts from means at A and B; by iteration 3 the assignment is stable and the means (X) sit at the centres of the two natural groups.

4.2 Initialization matters

Different starting means can converge to different local minima. On the standardized Wine data, 50 runs of k-means ($K = 3$), each from one random start, ended with WCSS between 1,277.9 and 1,617.4, and 27 of the 50 runs finished above the best value. Their agreement with the cultivars ranged from ARI 0.322 to 0.915. Two standard remedies, both defaults in scikit-learn:

- **k-means++** (Arthur and Vassilvitskii, 2007) picks the first centre at random and each later centre with probability proportional to its squared distance from the nearest centre already chosen, so the starting centres are spread out.
- **Multiple restarts** (`n_init`): run the algorithm several times and keep the solution with the lowest WCSS.

4.3 What k-means assumes

Because every point goes to its nearest centre, the boundary between two clusters is a straight line (the clusters form a Voronoi partition). k-means therefore works best when clusters are roughly spherical, of similar spread, and of similar size. It fails on elongated, curved, or nested shapes (Figure 4), and every point, including outliers, is forced into some cluster.

4.4 k-modes: the categorical equivalent

k-modes (Huang, 1998) keeps the k-means loop and changes two ingredients: the distance between two records is the **number of attributes on which they disagree**, and each cluster centre is the **mode** (most frequent value) of each attribute among the cluster's members.

1. Pick K records at random (or by judgement) as the initial modes.

2. Assign each record to the mode with the fewest mismatches.
3. Recompute each cluster's mode, attribute by attribute.
4. Repeat steps 2–3 until no record changes cluster.

Worked Example: k-Modes by Hand ($K = 2$)

Five patients with three categorical attributes (Smoker, Blood pressure, Activity): P_1 (Y, High, Low), P_2 (Y, High, Med), P_3 (N, Normal, High), P_4 (N, Normal, Med), P_5 (Y, Normal, Low). Initial modes: $M_1 = P_1$, $M_2 = P_3$.

Mismatch counts to (M_1, M_2): $P_1\,(0, 3)$, $P_2\,(1, 3)$, $P_3\,(3, 0)$, $P_4\,(3, 1)$, $P_5\,(1, 2)$. Clusters: $\{P_1, P_2, P_5\}$ and $\{P_3, P_4\}$.

New modes: cluster 1 has Smoker = Y (3 of 3), BP = High (2 of 3), Activity = Low (2 of 3), i.e. (Y, High, Low), unchanged. Cluster 2 has (N, Normal, High/Med); Activity is a tie, broken by keeping the current value High, so the mode is unchanged. No reassignment occurs; the algorithm has converged. Note that ties are common with categorical data, so implementations need an explicit tie-breaking rule.

5 Hierarchical Clustering

Hierarchical clustering builds a whole family of nested clusterings instead of one partition.

- **Agglomerative** ("bottom up"): every observation starts as its own cluster, and the two closest clusters are merged repeatedly until one cluster remains.
- **Divisive** ("top down"): all data starts as one cluster that is split recursively. It is less common because there are many more ways to split than to merge.

The merge history is drawn as a **dendrogram**. The height of each join is the distance at which the two groups merged; cutting the tree with a horizontal line at some height gives a flat clustering, and the number of vertical lines the cut crosses is the number of clusters. A long vertical gap before the next merge suggests a natural number of groups. The lecture's example groups European countries by their protein consumption; the same idea applied to wines is shown in Figure 2.

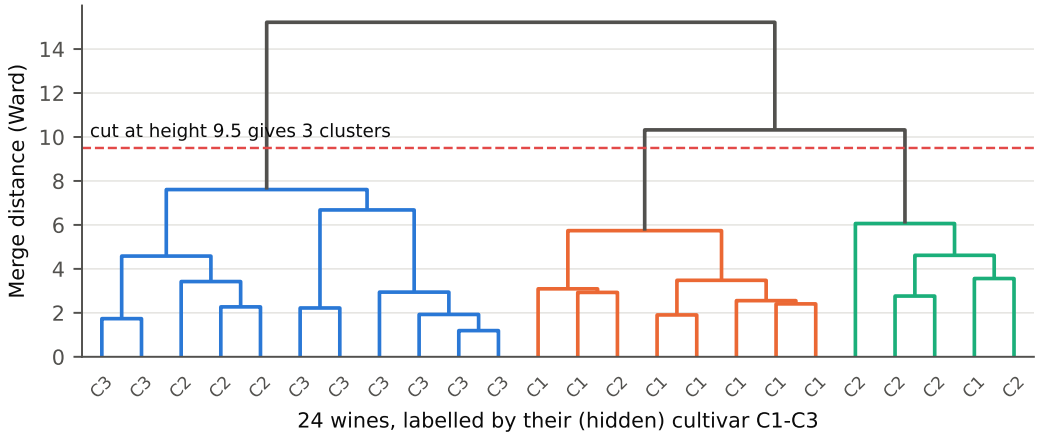


Figure 2: Ward dendrogram for 24 standardized wines (8 per cultivar). The cultivar labels were not used to build the tree. Cutting at height 9.5 gives three clusters; cultivars C1 and C3 are recovered almost cleanly, while C2 wines are spread across all three branches.

To merge *clusters* we need a distance between *groups* of points, called the **linkage**:

Table 3: Common linkage rules.

Linkage	**Distance between groups A and B**	**Behaviour**
Single	Smallest pairwise distance	Follows chains; finds curved shapes but can link everything through a few bridging points
Complete	Largest pairwise distance	Compact clusters; sensitive to outliers
Average	Mean pairwise distance	A compromise between single and complete
Ward (Ward, 1963)	Increase in WCSS caused by the merge	Compact, similar-sized clusters; the hierarchical analogue of k-means

On the standardized Wine data with three clusters, Ward linkage reaches ARI 0.790 with cluster sizes 58/56/64. Single linkage gives cluster sizes 174/3/1 and ARI -0.007: it chained almost every wine into one cluster and split off a few outliers. Hierarchical clustering needs only pairwise distances, so it works with any distance (including Gower), but storing all pairwise distances costs $O(n^2)$ memory, which limits it to tens of thousands of rows.

6 Gaussian Mixture Models

6.1 From one Gaussian to a mixture

The name says it all. A **Gaussian** (normal) distribution models a continuous variable such as height, weight, or yearly sales:

$$X \sim \mathcal{N}(\mu, \sigma^2), \qquad p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\!\left(-\tfrac{1}{2}\left(\tfrac{x-\mu}{\sigma}\right)^2\right).$$

A **mixture** has more than one component. Suppose we weigh students from grades 5, 6, and 10 without recording the grade. Each grade has its own normal distribution, and the pooled data follow

$$p(x) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x \mid \mu_k, \sigma_k^2), \qquad 0 \le \pi_k \le 1, \quad \sum_k \pi_k = 1,$$

where π_k is the share of the data from component k. With p features each component becomes a multivariate Gaussian $\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ whose covariance $\boldsymbol{\Sigma}_k$ lets the cluster be an ellipse of any orientation, not just a circle.

6.2 A latent-variable (generative) view

Introduce a hidden indicator $\mathbf{z}$ with a 1-of-K encoding: $z_k = 1$ if the point came from component k, and 0 otherwise. The model says: first draw a component with probability $p(z_k = 1) = \pi_k$, then draw $\mathbf{x}$ from that component. In graphical-model notation this is $Z \to X$, and

$$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})\, p(\mathbf{x} \mid \mathbf{z}), \qquad p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Because it models how the data are generated (the joint distribution), a GMM is a *generative* model, in contrast to the discriminative classifiers in the Classification reading. Its parameters are $\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$ for all K components.

6.3 Soft assignments with Bayes' rule

Given the parameters, Bayes' rule gives the probability that a point came from each component, called its **responsibility**:

$$\gamma_k(\mathbf{x}) = p(z_k = 1 \mid \mathbf{x}) = \frac{\pi_k \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

k-means and hierarchical clustering give **hard** assignments: each point belongs to exactly one cluster. A GMM gives **soft** assignments that show how confident the assignment is.

Worked Example: Which Grade Is This Student In?

Let grade 5 weights be $\mathcal{N}(38, 4^2)$, grade 6 $\mathcal{N}(42, 4.5^2)$, and grade 10 $\mathcal{N}(60, 6^2)$ kg, with equal weights $\pi_k = 1/3$. For a student weighing $x = 40$ kg the three densities times π_k are 0.02934, 0.02677, and 0.00009. Dividing by their sum gives responsibilities 0.52, 0.48, and 0.00: almost certainly grade 5 or 6, and honestly undecided between them. A 50 kg student gets 0.03, 0.51, 0.46: a coin flip between grades 6 and 10. A hard clustering would report a single grade in both cases and hide the uncertainty.

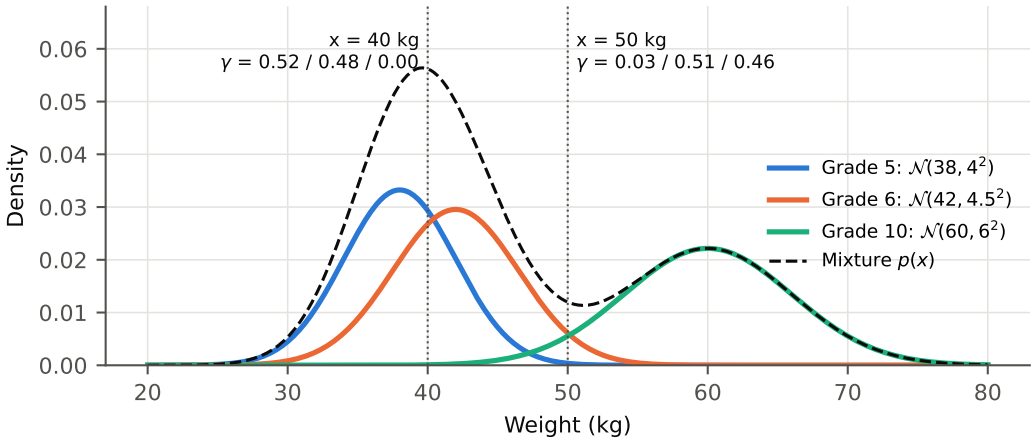


Figure 3: Three-component mixture for the worked example. Coloured curves are $\pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$; the dashed curve is their sum, the density of the pooled data. Responsibilities γ (grade 5 / 6 / 10) are shown for $x = 40$ and $x = 50$ kg.

6.4 Learning the parameters: maximum likelihood and EM

In practice the parameters are unknown. We start from a random (or k-means-based) initialization and maximize the log-likelihood of the N data points,

$$\ln p(X \mid \boldsymbol{\theta}) = \sum_{n=1}^{N} \ln \Big\{ \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \Big\}, \qquad \boldsymbol{\theta} = \{\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}\}.$$

Because of the sum inside the logarithm there is no closed-form maximizer. Instead we use the **EM** (Expectation–Maximization) algorithm (Dempster et al., 1977):

- **E-step:** with the current parameters, compute the responsibilities γ_{nk} for every point.
- **M-step:** update each component using the points weighted by their responsibilities: $N_k = \sum_n \gamma_{nk}$, $\pi_k = N_k/N$, $\boldsymbol{\mu}_k = \frac{1}{N_k}\sum_n \gamma_{nk}\mathbf{x}_n$, and $\boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_n \gamma_{nk}(\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$.

Repeat until the log-likelihood stops improving. Each iteration cannot decrease the likelihood, but, as with k-means, the result is a local optimum, so several restarts help. (Gradient-based optimization is an alternative, but EM is the standard choice.) k-means is in fact a limiting case of EM for a GMM with equal, spherical covariances and responsibilities rounded to 0 or 1 (Bishop, 2006).

Why Use a GMM?

(1) Soft assignments $p(Z \mid X)$ quantify how sure each assignment is. (2) Full covariances allow elliptical, tilted clusters of different sizes. (3) The fitted model is a probability distribution $p(X, Z)$ over the feature space, so it can score how unusual a new point is (anomaly detection), generate synthetic data, or serve as a building block in a larger probabilistic model.

Each full covariance matrix has $p(p+1)/2$ free parameters, so in many dimensions with few rows the model can overfit or become numerically unstable. scikit-learn's `covariance_type` option trades flexibility for stability: `full`, `tied` (one shared matrix), `diag` (axis-aligned ellipses), or `spherical` (circles, closest to k-means).

7 DBSCAN: Density-Based Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise; Ester et al., 1996) defines a cluster as a contiguous region of high point density, separated from other clusters by regions of low density. It has two parameters:

- ε (`eps`): the radius of the neighbourhood around each point;
- minPts (`min_samples`): the minimum number of points within ε for a region to count as dense.

These give three kinds of points. A **core point** has at least minPts points (itself included) within ε. A **border point** is within ε of a core point but is not itself core. A **noise point** is neither.

The algorithm picks an unvisited point. If it is a core point, a new cluster starts: all points within ε join, and the neighbourhood search is repeated from every new core point, so the cluster grows outward until no more points can be reached. Border points join the cluster but do not extend it. Points never reached are labelled noise (-1 in scikit-learn).

Strengths. DBSCAN does not need K, finds clusters of arbitrary shape (it separates the two interlocking moons in Figure 4 perfectly), and labels outliers as noise instead of forcing them into a cluster.

Weaknesses. One global ε cannot fit clusters of very different density: in the bottom row of Figure 4 the sparse cluster is mostly labelled noise. It also suffers in many dimensions, where distances become similar (Section 10). On the 13-dimensional standardized Wine data with minPts $= 26$, $\varepsilon = 2.0$ labels all 178 wines as noise, $\varepsilon = 2.5$ labels 130 as noise, and $\varepsilon = 3.0$ finds only 2 clusters. After reducing to two principal components DBSCAN behaves better but merges two touching cultivars into one cluster (ARI 0.438 at $\varepsilon = 0.8$), because nothing in the density separates them.

Choosing ε and minPts

A common rule of thumb is minPts $\approx 2\times$(number of features), and at least 4 for 2-D data (Schubert et al., 2017). Then compute, for every point, the distance to its minPts-th nearest neighbour, sort these distances, and plot them (a *k-distance plot*). Choose ε near the "knee" where the curve turns sharply upward: points to the right of the knee are in sparse regions and will become noise. HDBSCAN, available in scikit-learn, removes the need for a single ε by considering all densities at once.

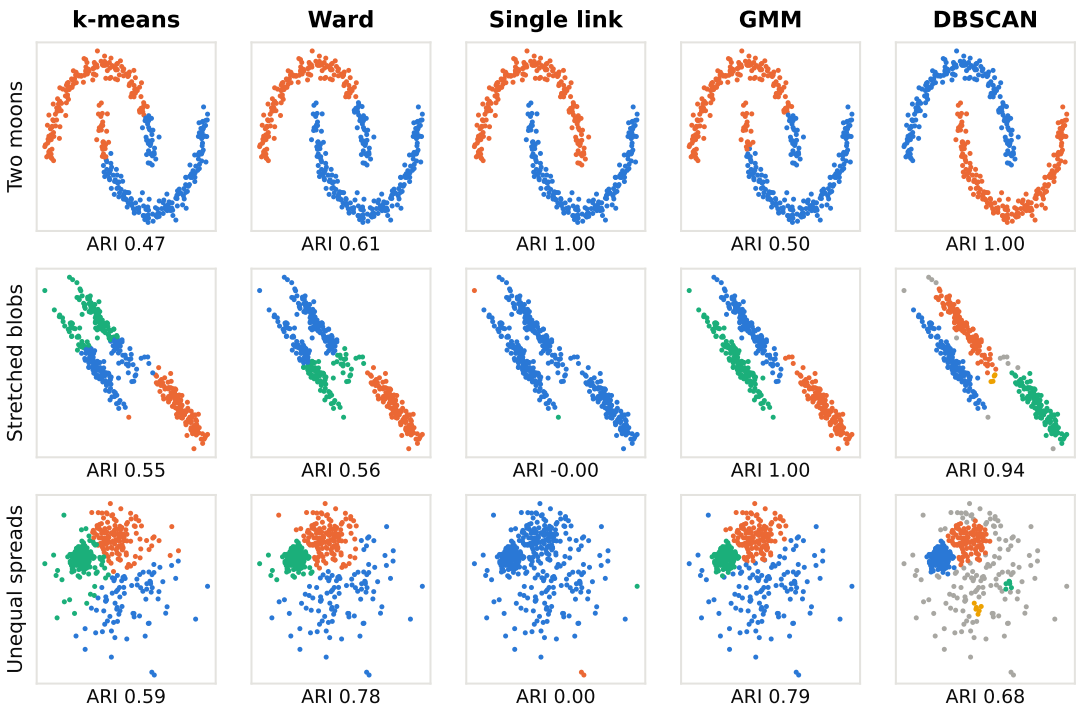


Figure 4: Five algorithms on three synthetic shapes (data standardized; K given to all but DBSCAN). No algorithm wins everywhere: single linkage and DBSCAN follow the moons; the GMM's full covariances capture stretched blobs; Ward and the GMM handle unequal spreads best. Compare with the scikit-learn clustering gallery shown in the lecture.

8 How Many Clusters?

How many clusters best describe a set of points? Looking at three well-separated clouds, one cluster is too few and four splits a real group, so three seems right. In more than two dimensions we cannot look, so we use criteria.

8.1 The elbow method (explained variance)

Split the total variation of the data into the part *between* clusters and the part *within* them:

$$\underbrace{\sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2}_{\text{TV (total)}} = \underbrace{\sum_j n_j \|\boldsymbol{\mu}_j - \bar{\mathbf{x}}\|^2}_{\text{BCV (between clusters)}} + \underbrace{\sum_j \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2}_{\text{WCSS (within clusters)}}.$$

The percentage of variance explained is $100 \times \text{BCV}/\text{TV}$. This is the same ratio of between-group to total variation that underlies the one-way ANOVA F-test. Adding clusters always raises it (with $K = n$ it reaches 100%), so we do not look for its maximum. Instead we plot it, or equivalently WCSS, against K and look for the **elbow**: the point after which an extra cluster buys little. The same reasoning is used to choose the number of PCA components (Section 11).

8.2 The silhouette score

For each point i, let a_i be its mean distance to the other points in its own cluster (cohesion) and b_i its mean distance to the points of the *nearest other* cluster (separation). Its silhouette coefficient (Rousseeuw, 1987) is

$$s(i) = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1].$$

Values near $+1$ mean the point is well inside its cluster; near 0 it sits on a boundary between overlapping clusters; negative values suggest it is in the wrong cluster. The **silhouette score** is the mean of $s(i)$ over all points; choose the K that maximizes it. For example, $a_i = 1.2$ and $b_i = 3.0$ give $s(i) = (3.0 - 1.2)/3.0 = 0.60$, while $a_i = 2.0$ and $b_i = 1.5$ give $s(i) = (1.5 - 2.0)/2.0 = -0.25$.

8.3 Information criteria for GMMs

Because a GMM has a likelihood, we can use the Bayesian information criterion, $\mathrm{BIC} = -2 \ln \hat{L} + m \ln n$, where m is the number of free parameters (Schwarz, 1978). Lower is better: the first term rewards fit, the second penalizes complexity.

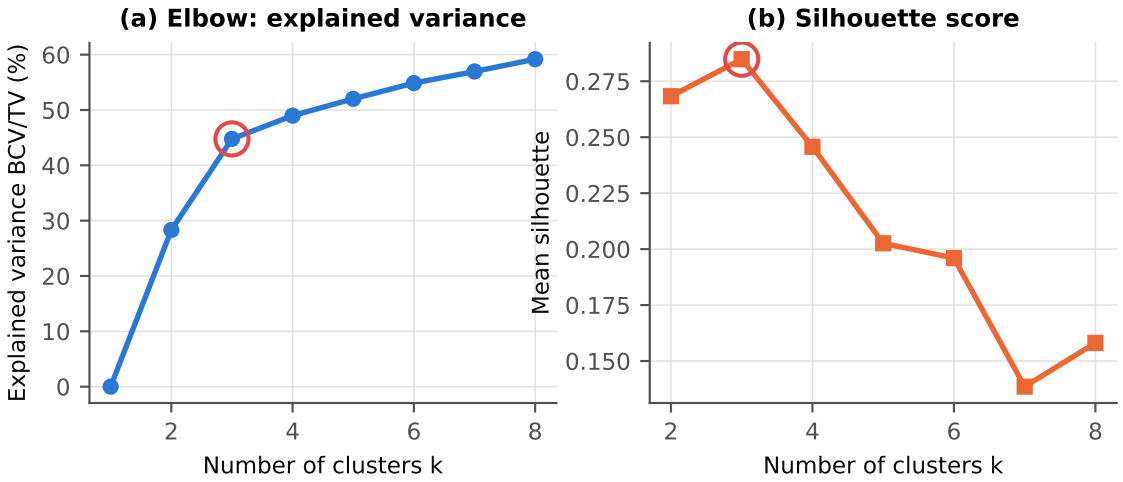


Figure 5: Choosing K for k-means on the standardized Wine data. (a) Explained variance rises from 28.3% ($K = 2$) to 44.8% ($K = 3$), then only 4.2 points more for $K = 4$: an elbow at 3. (b) The silhouette score peaks at $K = 3$ (0.285).

On Wine, both the elbow and the silhouette point to $K = 3$, which matches the three cultivars. BIC tells a more cautious story: with full covariances it prefers $K = 2$ (BIC 5,608 versus 5,806 for $K = 3$), because each extra 13-dimensional component adds 104 parameters for only 178 rows; with diagonal covariances it prefers $K = 4$. Criteria can disagree, and they answer slightly different questions.

Common Pitfall

No criterion "discovers" the true number of clusters. The elbow is often ambiguous, silhouette favours compact round clusters, and BIC depends on the assumed covariance structure. Use several criteria, check whether the clusters are stable when you resample the data, and ask whether the groups are useful for the decision at hand. Sometimes the honest answer is that the data have no strong cluster structure at all.

9 Evaluating and Interpreting Clusters

There are three complementary ways to judge a clustering:

1. **Internal validation** uses only the data and the cluster labels: WCSS, silhouette, BIC (Section 8). It measures how compact and separated the clusters are, which is not the same as how meaningful they are.
2. **External validation** compares the clusters with labels that were *not* used to build them, such as a known diagnosis or cultivar. This is mostly possible in teaching and benchmarking, where we deliberately hide a label.
3. **Stability** asks whether similar clusters appear when you rerun with different seeds, bootstrap samples, or slightly different features. Clusters that vanish under small changes should not be reported as findings.

The adjusted Rand index. Cluster numbers are arbitrary (cluster "1" in one run may be cluster "3" in another), so we cannot compare labels directly. The Rand index instead looks at every *pair* of points and asks whether the two partitions agree on keeping the pair together or apart. The **adjusted** Rand index (ARI; Hubert and Arabie, 1985) rescales this so that random labelling scores about 0 and identical partitions score 1; negative values mean worse than chance. ARI ignores the label names, so permuting cluster numbers does not change it. In scikit-learn: `adjusted_rand_score(y_true, labels)`.

Interpreting clusters. Once you have clusters, profile them: compare each cluster's feature means (or modes) with the overall data, look at cluster sizes, and plot the clusters in two principal components. A cluster becomes knowledge only when it can be described in domain terms and acted upon.

Real-World Case Study

Sepsis phenotypes from routine clinical data. Sepsis is a heterogeneous syndrome, and treatments tested on "all sepsis patients" have often failed in trials. Seymour et al. (2019) applied consensus k-means clustering to 29 routinely collected clinical variables (vital signs, laboratory values, demographics) from roughly 20,000 patients and identified four phenotypes, named α, β, γ, and δ, that differed in organ dysfunction, inflammation markers, and mortality. The phenotypes were reproduced in separate validation cohorts, and simulations using data from earlier trials suggested that treatment benefit could differ by phenotype. The study illustrates the full unsupervised workflow: careful preprocessing, several criteria for K, stability checks, validation on new data, and clinical interpretation before any claim is made.

10 The Curse of Dimensionality

Clustering and nearest-neighbour methods rely on distances, and distances behave badly when there are many features. Picture the heights of the students in a class, min–max normalized to $[0, 1]$ and split into 3 equal bins. Nobody in the class is half the height of the tallest person, so the students fill roughly one bin: about a third of the space (the lecture estimates ∼40%). Add weight as a second feature with 3 bins: there are now $3 \times 3 = 9$ cells and the same students fill roughly one of them, about 11% of the space (lecture: ∼10%). Equivalently, the empty space grew from about

60% to about 90%. With four features there are 81 cells and the class covers about 1%. The space to fill grows exponentially with the number of features, as 3^d (Bellman, 1961).

Two consequences follow. First, data become **sparse**: with a fixed number of rows, most of the feature space is empty, and models have little evidence about most of it. Second, **distances concentrate**: the nearest and farthest neighbours of a point end up at almost the same distance (Beyer et al., 1999). For 500 uniformly random points, the ratio (farthest − nearest)/nearest for a random query point falls from 126 with 2 features to 0.34 with 100 features and 0.15 with 1,000 (Figure 6b). When "near" and "far" look alike, clustering has little to work with.

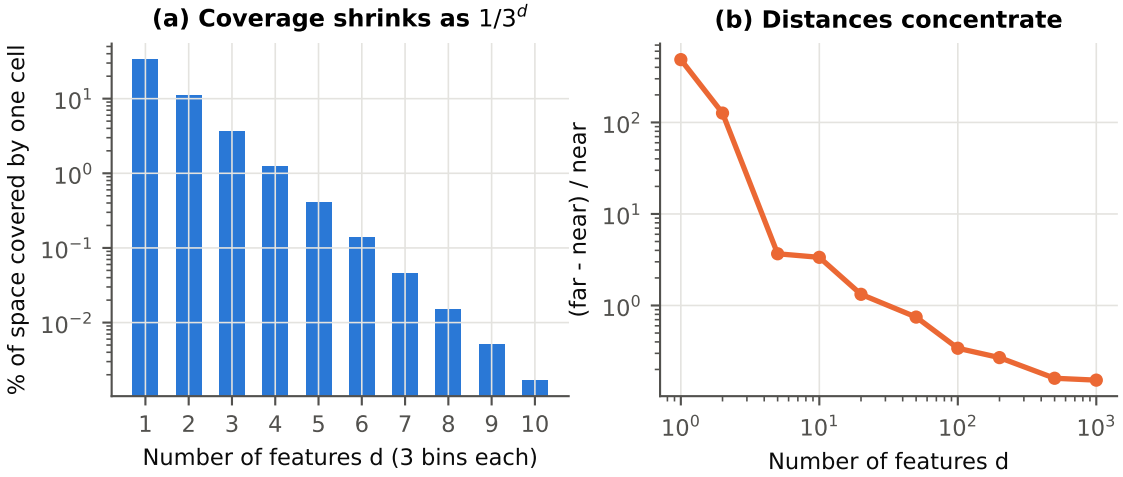


Figure 6: (a) With 3 bins per feature, one cell covers $100/3^d$ percent of the space (log scale). (b) Relative contrast between the farthest and nearest of 500 random points, by number of features (log–log).

The Feature Selection reading answered the curse by *keeping* a subset of the original features. **Dimensionality reduction** instead *constructs* a small number of new features: given data with $|D|$ features, we compress them to $|d| < |D|$ features. Linear methods include principal component analysis (PCA) and the singular value decomposition (SVD); nonlinear and neural-network methods include t-SNE, UMAP, autoencoders, and restricted Boltzmann machines.

11 Principal Component Analysis

11.1 Covariance: how two features vary together

The covariance of two features x and y measures whether they move together:

$$\operatorname{cov}(x, y) = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \bar{x})(y_i - \bar{y}).$$

It is positive when large x goes with large y, negative when large x goes with small y, and near zero when there is no linear relationship. For p features the **covariance matrix S** is $p \times p$: the diagonal holds each feature's variance and the off-diagonal entry (j, l) holds $\operatorname{cov}(x_j, x_l)$. Because $\operatorname{cov}(x_j, x_l) = \operatorname{cov}(x_l, x_j)$, the matrix is **symmetric**. For example, $\begin{bmatrix} 4 & 3 \\ 3 & 4 \end{bmatrix}$ describes a cloud tilted upward, $\begin{bmatrix} 4 & -3 \\ -3 & 4 \end{bmatrix}$ one tilted downward, and $\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$ a round cloud.

11.2 The idea: the data's own coordinate system

Take a cloud of points from a 2-D Gaussian in which x and y are correlated. The cloud is an ellipse tilted away from the x and y axes. It has its own natural axes, x' along the long direction of the ellipse and y' across it. These are the **principal components**. Describing each point by its coordinate on x' alone loses only the small spread along y', so we can go from two features to one while keeping most of the information.

11.3 PCA step by step

1. Start with the data matrix D (m rows, n features).
2. Standardize each feature (StandardScaler), so that features measured in large units do not dominate.
3. Estimate the covariance matrix A ($n \times n$).
4. Compute the eigenvalues and eigenvectors of A by solving $A\mathbf{v} = \lambda\mathbf{v}$, i.e. $(A - \lambda I)\mathbf{v} = \mathbf{0}$, which has non-zero solutions only when $\det(A - \lambda I) = 0$. An $n \times n$ symmetric matrix gives n real eigenvalues and n orthogonal eigenvectors.
5. Sort the eigenvectors by decreasing eigenvalue. Eigenvector $\mathbf{u}_j$ is the j-th principal component, and λ_j is the variance of the data along it.
6. Keep the first d components and project: the new features are $\mathbf{Z} = D_{\text{std}}\,[\mathbf{u}_1 \cdots \mathbf{u}_d]$.

In practice libraries (Pedregosa et al., 2011) compute the same result through the singular value decomposition of the standardized data, which is more numerically stable than forming A.

Worked Example: Eigenvalues of a 2×2 Covariance Matrix

Let $C = \begin{bmatrix} 4 & 3 \\ 3 & 7 \end{bmatrix}$. Then $\det(C - \lambda I) = (4 - \lambda)(7 - \lambda) - 9 = \lambda^2 - 11\lambda + 19 = 0$, so

$$\lambda = \tfrac{11 \pm \sqrt{121 - 76}}{2} = \tfrac{11 \pm 6.708}{2}, \qquad \lambda_1 = 8.854, \quad \lambda_2 = 2.146.$$

The first eigenvector is $\mathbf{u}_1 \approx (0.526, 0.851)$, pointing up and to the right along the cloud; $\mathbf{u}_2 \approx (-0.851, 0.526)$ is perpendicular to it. PC1 explains $8.854/11 = 80.5\%$ of the total variance (the total is the trace, $4 + 7 = 11$). When C is diagonal, such as $\begin{bmatrix} 7 & 0 \\ 0 & 1 \end{bmatrix}$, the eigenvectors are simply the original axes and the eigenvalues are the two variances.

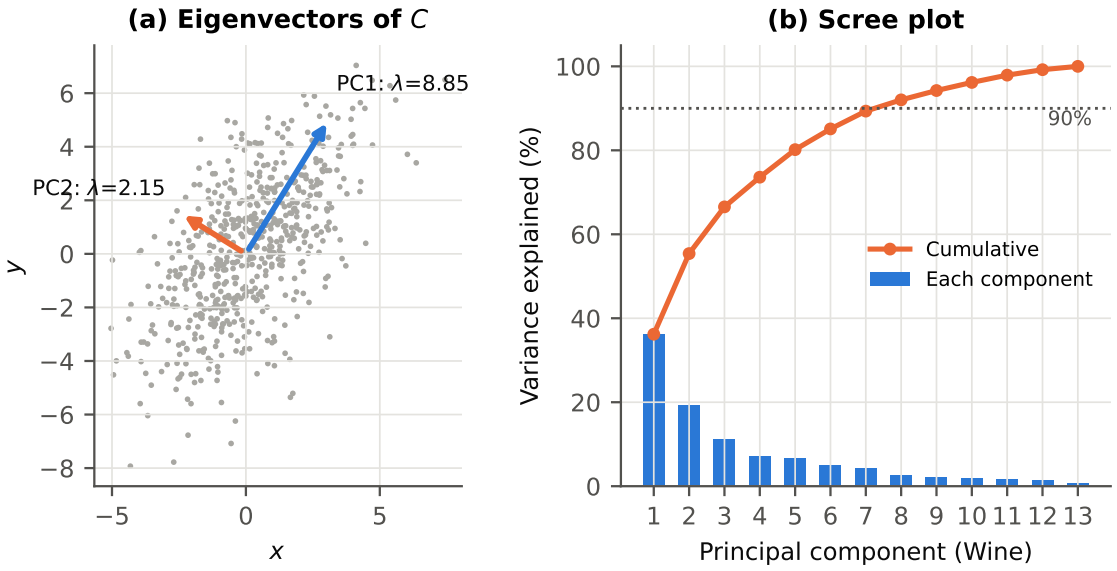


Figure 7: (a) 600 points drawn with covariance C from the worked example; arrows are the eigenvectors scaled by $2\sqrt{\lambda}$. (b) Scree plot for the 13 standardized Wine features: PC1 explains 36.2%, PC2 19.2%; 5 components reach 80% and 8 reach 90%.

11.4 What PCA optimizes

Project each 2-D point $\mathbf{x}_n$ onto a line through the data mean with direction $\mathbf{u}_1$. The projected point $\tilde{\mathbf{x}}_n$ is its 1-D representation, and the segment from $\mathbf{x}_n$ to $\tilde{\mathbf{x}}_n$ is the information lost. Two equivalent goals give the same line:

- **Maximize variance**: spread the projected points $\tilde{\mathbf{x}}_n$ out as much as possible along the line.
- **Minimize error**: make the total squared length of the segments from $\mathbf{x}_n$ to $\tilde{\mathbf{x}}_n$ (the reconstruction error) as small as possible.

They are equivalent because, by Pythagoras, each point's squared distance from the mean equals its squared projection plus its squared error; the left side is fixed, so maximizing one term minimizes the other (Pearson, 1901). Formally, the variance of the projected data is

$$\frac{1}{N}\sum_{n=1}^{N}\{\mathbf{u}_1^\top \mathbf{x}_n - \mathbf{u}_1^\top \bar{\mathbf{x}}\}^2 = \mathbf{u}_1^\top \mathbf{S}\mathbf{u}_1, \qquad \mathbf{S} = \frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top.$$

Without a constraint we could inflate this by making $\mathbf{u}_1$ longer, so we require $\mathbf{u}_1^\top \mathbf{u}_1 = 1$ and maximize the Lagrangian $\mathbf{u}_1^\top \mathbf{S}\mathbf{u}_1 + \lambda(1 - \mathbf{u}_1^\top \mathbf{u}_1)$. Setting its gradient to zero gives $\mathbf{S}\mathbf{u}_1 = \lambda \mathbf{u}_1$: the optimal direction is an eigenvector, and the projected variance equals λ, so we pick the eigenvector with the largest eigenvalue (Bishop, 2006). Each later component maximizes the remaining variance subject to being orthogonal to the earlier ones. The same derivation generalizes from 2 to 1 dimension to any reduction from D to M dimensions.

11.5 How many components, and what do they mean?

The **explained variance ratio** $\lambda_j / \sum_l \lambda_l$ tells how much of the total variance component j keeps. Plot it with its cumulative sum (a *scree plot*, Figure 7b) and either look for an elbow, exactly as in

Section 8, or keep enough components to reach a target such as 80% or 90%. For Wine, the first three eigenvalues are 4.73, 2.51, and 1.45. If all 13 standardized features were unrelated each would have eigenvalue 1, so the first component carries the information of almost five original features.

The entries of each eigenvector, called **loadings**, say what the component means. PC1 for Wine loads most heavily on flavanoids (0.423), total phenols (0.395), the OD280/OD315 ratio (0.376), and proanthocyanins (0.313): it is essentially a "phenolic content" axis. Loadings can be read only because the features were standardized; without scaling, PC1 for Wine would simply be proline, the feature with the largest raw variance.

PCA and Clustering Work Together

Running k-means on just the first two Wine components (36.2% + 19.2% = 55.4% of the variance) gives ARI 0.895, essentially the same as on all 13 features (0.897). PCA removed redundancy and noise while keeping the structure that separates cultivars, and it gave us a 2-D picture we can actually look at. Fit the scaler and PCA inside a pipeline so that, in any later supervised use, they are learned from training data only (the leakage rule from the Feature Selection reading).

PCA has limits. It finds only *linear* directions, so a curved structure (such as the moons in Figure 4) cannot be unrolled. High variance is not always high importance: a direction that separates two rare classes may carry little variance. And components mix all original features, which makes them harder to explain than a selected subset. Jolliffe and Cadima (2016) review these issues and modern variants such as sparse and robust PCA.

12 Nonlinear Embeddings for Visualization: t-SNE and UMAP

When structure is nonlinear, two methods dominate for *visualizing* high-dimensional data in 2-D.

- **t-SNE** (van der Maaten and Hinton, 2008) converts distances into probabilities that two points are neighbours, both in the original space (with Gaussian kernels) and in the 2-D map (with a heavy-tailed Student-t kernel). It then moves the 2-D points to make the two sets of probabilities match. The *perplexity* parameter (typically 5–50) sets roughly how many neighbours each point considers.
- **UMAP** (McInnes et al., 2018) builds a weighted k-nearest-neighbour graph in the original space and lays out a 2-D graph with similar connectivity. Its main parameters are `n_neighbors` (local versus global emphasis) and `min_dist` (how tightly points may pack). It is usually faster than t-SNE and tends to keep more of the global arrangement, and it can transform new points.

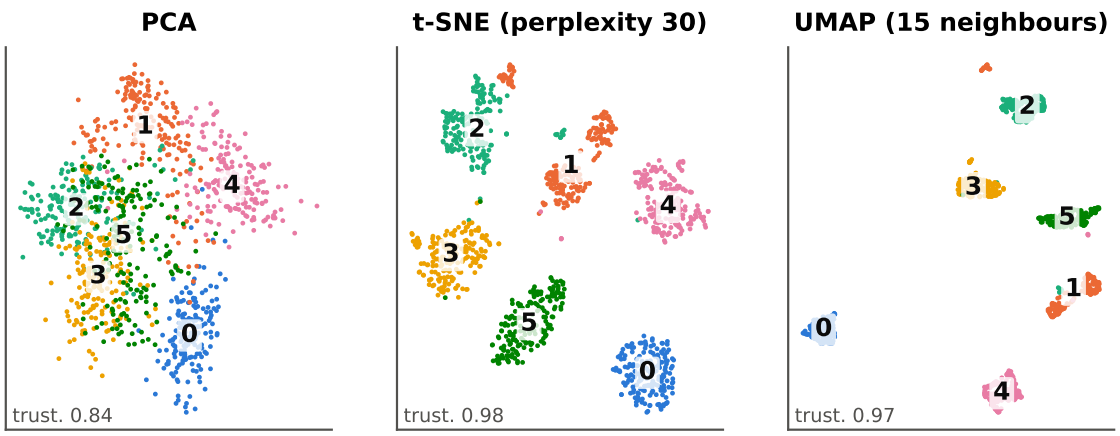


Figure 8: PCA, t-SNE, and UMAP views of 1,083 handwritten digits (0–5, 64 pixel features, standardized). Trustworthiness measures how well each map's 10 nearest neighbours were true neighbours in the 64-D space (1 = perfect): PCA 0.84, t-SNE 0.98, UMAP 0.97. Colours and digit labels were added afterwards for reading the plot.

Figure 8 shows the difference. Two linear components (PCA) leave the digit groups overlapping; t-SNE and UMAP pull them into separate islands while preserving local neighbourhoods much better. These maps are powerful exploratory tools, but they must be read with care (Wattenberg et al., 2016):

Common Pitfall: Over-Reading t-SNE and UMAP Plots

- Cluster *sizes* and the *distances between* islands in a t-SNE map are largely meaningless; the method preserves who is near whom, not how far apart groups are.
- Different perplexity or `n_neighbors` values, or even different random seeds, can produce visibly different pictures. Try several before believing a pattern.
- Both methods can create apparent clusters from data that have none. Confirm any structure with a clustering on the original (or PCA-reduced) features and with the criteria of Section 8.
- Use them for visualization, not as a general preprocessing step: unlike PCA, t-SNE has no transform for new data, and neither method's axes have an interpretation.

13 Python Implementation

The listing below condenses the companion script. It uses the Wine data, keeps the cultivar label aside, and uses it only at the end for external validation.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
```

```python
from sklearn.metrics import silhouette_score, adjusted_rand_score

wine = load_wine(as_frame=True)
X, y_hidden = wine.data, wine.target        # y_hidden: NEVER used for fitting
Xs = StandardScaler().fit_transform(X)      # scale before any distance method

# 1) Choose k: explained variance (elbow) and silhouette
TV = ((Xs - Xs.mean(axis=0)) ** 2).sum()
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(Xs)
    print(k, round(100 * (TV - km.inertia_) / TV, 1),
          round(silhouette_score(Xs, km.labels_), 3))

# 2) Compare algorithms at k = 3
models = {"k-means": KMeans(3, n_init=10, random_state=0),
          "Ward":    AgglomerativeClustering(3, linkage="ward"),
          "GMM":     GaussianMixture(3, n_init=5, random_state=0)}
for name, m in models.items():
    labels = m.fit_predict(Xs)
    print(name, silhouette_score(Xs, labels), adjusted_rand_score(y_hidden, labels))

# 3) PCA: variance explained and loadings
pca = PCA().fit(Xs)
print(np.cumsum(pca.explained_variance_ratio_)[:8])

# 4) Scale -> PCA(2) -> k-means as one pipeline
pipe = make_pipeline(StandardScaler(), PCA(n_components=2, random_state=0),
                     KMeans(n_clusters=3, n_init=10, random_state=0))
labels2 = pipe.fit_predict(X)
print("ARI on 2 PCs:", adjusted_rand_score(y_hidden, labels2))
```

Table 4: Clustering the standardized Wine data with $K = 3$ (scikit-learn 1.9, `random_state`=0). Silhouette is internal; ARI against the hidden cultivar is external.

Method	Silhouette	ARI vs. cultivar	Cluster sizes
k-means (raw, unscaled features)	—	0.371	—
k-means (standardized)	0.285	**0.897**	65 / 51 / 62
GMM, full covariance	0.284	0.880	51 / 63 / 64
Agglomerative, Ward	0.277	0.790	58 / 56 / 64
Agglomerative, single	0.183	−0.007	174 / 3 / 1
k-means on 2 principal components	—	0.895	—

Three lessons stand out. Scaling more than doubled agreement with the cultivars. The silhouette ranking roughly follows the ARI ranking here, but the two measure different things, and only one wine received a GMM responsibility below 0.9, so the GMM's soft assignments were nearly hard on this well-separated data. Finally, a method whose assumptions do not fit (single linkage on compact groups) can fail completely while still returning a perfectly valid-looking set of labels.

14 Case Study: Do Natural Groups Match the Diagnosis?

The Breast Cancer Wisconsin Diagnostic data (Street et al., 1993), used in the Classification reading, describe 569 tumours with 30 features computed from digitized images of cell nuclei (radius, texture,

concavity, and so on). Suppose we ignore the diagnosis and simply ask whether the tumours fall into natural groups.

After standardizing, PCA shows strong redundancy: PC1 alone explains 44.3% of the variance, PC1 and PC2 together 63.2%, and 7 of the 30 components reach 90%. This matches the Feature Selection reading's finding that many of these features are almost copies of each other (radius, perimeter, and area). The silhouette score is highest for $K = 2$ (0.343, versus 0.314 for $K = 3$ and 0.159 for $K = 4$), so we fit k-means with two clusters and only then compare with the hidden diagnosis:

Table 5: k-means ($K = 2$) clusters versus the hidden diagnosis.

	Benign	**Malignant**
Cluster 0	339	36
Cluster 1	18	176

The clusters align with the diagnosis far better than chance (ARI 0.654), and a two-component GMM, whose elliptical components fit the correlated features better, does better still (ARI 0.774). Yet 36 malignant tumours sit in the mostly benign cluster. For a screening application that is a serious failure; the logistic regression in the Classification reading, which *was* trained on the diagnosis, missed only 3 malignant tumours in its test set.

Lesson from the Case Study

Clusters capture the largest, most obvious structure in the features, here essentially "big, irregular nuclei versus small, regular ones." That structure often correlates with a label of interest, but clustering is not trying to predict the label and should not be used as a substitute for a classifier when labels exist. Use clustering to discover structure, build hypotheses, and create features; use supervised learning when you have a target to predict.

15 Common Pitfalls

1. **Clustering unscaled data.** The feature with the largest units decides the clusters (Wine ARI 0.371 raw versus 0.897 scaled).
2. **Trusting a single k-means run.** Results depend on the starting means; use k-means++ and several restarts and compare WCSS.
3. **Treating K as known.** Elbow, silhouette, and BIC can disagree; report the evidence, not just the chosen number.
4. **Using an algorithm whose notion of "cluster" does not fit the data**: k-means on curved or elongated shapes, single linkage on compact overlapping groups, DBSCAN with one ε on clusters of varied density.
5. **Forcing every point into a cluster.** Outliers distort k-means centroids; consider DBSCAN's noise label or a GMM's low-likelihood points.
6. **Clustering in very many dimensions.** Distances concentrate; reduce dimensions or select features first.

7. **Encoding categories as integers** (red = 1, green = 2, blue = 3) and running k-means, which invents an order and distances that do not exist. Use one-hot encoding, k-modes, k-prototypes, or Gower distance.
8. **Over-reading t-SNE/UMAP maps**: island sizes and gaps are not data, and apparent clusters need independent confirmation.
9. **Naming clusters too early.** A label such as "high-risk patients" is an interpretation that needs domain review and validation on new data.
10. **Leakage when unsupervised steps feed a supervised model.** Fit scalers, PCA, and clusterers on training data only, inside a pipeline.

16 Key Takeaways

- Unsupervised learning models $f(X)$ without a label. Its two main tasks are clustering (grouping rows) and dimensionality reduction (compressing columns).
- The features, their scaling, and the distance define similarity, and therefore define what any clustering can find.
- k-means minimizes WCSS by alternating assignment and update steps; it needs K, numeric data, and good initialization, and it prefers round clusters. k-modes and k-prototypes extend it to categorical and mixed data.
- Hierarchical clustering builds a dendrogram; the linkage rule controls the cluster shapes it favours.
- GMMs are generative, give soft assignments via Bayes' rule, are fitted by EM, and handle elliptical clusters.
- DBSCAN finds arbitrarily shaped dense regions and labels noise, but needs careful ε and struggles with varying density and high dimensions.
- Choose K with several criteria (explained variance elbow, silhouette, BIC), then check stability and usefulness. Evaluate with internal measures and, when hidden labels exist, ARI.
- High dimensions make data sparse and distances uninformative. PCA finds orthogonal directions of maximum variance (eigenvectors of the covariance matrix); keep components by explained variance.
- t-SNE and UMAP are excellent for seeing nonlinear neighbourhood structure but are visualization tools, not proof of clusters.

Wrap-Up: Connecting Back

The Regression and Classification readings learned from labels; this reading has shown what can be learned without them, and how much care that requires. The distance measures from the Advanced EDA reading are the foundation of every clustering algorithm here. The scaling rules from the Feature Scaling notebook decided whether k-means could find the Wine cultivars at all. PCA is the constructive counterpart of the Feature Selection reading: both fight the curse of dimensionality, one by keeping original features, the other by building new ones. The evaluation habits from the Hyper-parameter Optimization reading carry over too: choosing K is a model-selection problem, and

reporting the best of many criteria without checking stability is the unsupervised version of tuning on the test set. In your team project, unsupervised methods are most useful early, to explore and summarize linked data sets, and late, as engineered features (cluster indices, principal components) for a supervised model.

Review Questions

1. Explain the difference between learning $f(y \mid X)$ and $f(X)$. Give one health-related example of each.
2. Rerun the k-means worked example starting from means at C and D. Do you reach the same final clusters? How many iterations are needed?
3. Why can k-means not be applied directly to categorical data? Describe the two ingredients that k-modes changes, and compute the mismatch distance between (Y, Normal, Med) and (N, Normal, Low).
4. Compute the Gower distance between a 30-year-old non-smoker with BMI 21 and a 70-year-old smoker with BMI 33, using the ranges in Section 2.
5. Single linkage gave cluster sizes 174/3/1 on Wine. Explain the chaining effect that causes this. Which data shape in Figure 4 does single linkage handle best, and why?
6. In the grade-weight GMM, compute the responsibilities for a 45 kg student. Why might a soft assignment be more useful than a hard one in this setting?
7. Name two situations in which DBSCAN is preferable to k-means and one in which it is worse. How would you choose ε?
8. Show that explained variance BCV/TV can only increase as K grows. Why does this mean we look for an elbow rather than a maximum?
9. A point has $a_i = 0.8$ and $b_i = 0.9$. Compute its silhouette coefficient and interpret it.
10. The ARI of a clustering against hidden labels is 0.02. What does this mean? Could the clustering still be useful?
11. For $C = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}$, compute both eigenvalues and the share of variance explained by the first principal component.
12. Explain why maximizing projected variance and minimizing reconstruction error give the same first principal component.
13. A colleague shows a t-SNE map with two far-apart islands and concludes the two groups are "very different." What would you tell them, and what analysis would you run to check the claim?

Suggested Hands-on Exercise

Use the full handwritten digits data (`sklearn.datasets.load_digits`, 1,797 images, 64 pixel features, 10 digits). Keep the digit label aside for external validation only.

1. **Reduce.** Standardize the pixels and fit PCA. How many components are needed for 80% and 90% of the variance? Plot the first two components and the scree plot. Reshape the first component's loadings to 8×8 and display them as an image: which pixels matter?
2. **Cluster.** Run k-means for $K = 2, \ldots, 15$ on (a) all 64 standardized features and (b) the PCA components reaching 90%. Plot explained variance and silhouette against K. Does either criterion point to 10? Fit a GMM and choose K by BIC (try `covariance_type "diag"` and `"full"`).
3. **Validate and visualize.** For $K = 10$, report the ARI of k-means, Ward, and GMM against the true digits. Produce t-SNE (perplexity 5, 30, and 50) and UMAP maps coloured by *your* cluster labels. Which digits are most often confused, and is that visible in the maps? Write one paragraph on what you would and would not conclude from the maps alone.

References

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms* (pp. 1027–1035). Society for Industrial and Applied Mathematics.

Bellman, R. (1961). *Adaptive control processes: A guided tour*. Princeton University Press.

Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is "nearest neighbor" meaningful? In *Database Theory — ICDT'99* (pp. 217–235). Springer. https://doi.org/10.1007/3-540-49257-7_15

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, *39*(1), 1–22. https://doi.org/10.1111/j.2517-6161.1977.tb01600.x

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)* (pp. 226–231). AAAI Press.

Gower, J. C. (1971). A general coefficient of similarity and some of its properties. *Biometrics*, *27*(4), 857–871. https://doi.org/10.2307/2528823

Han, J., Pei, J., & Tong, H. (2023). *Data mining: Concepts and techniques* (4th ed.). Morgan Kaufmann.

Hasan, K. (2026). *CIS 635 Knowledge Discovery and Data Mining: Course reading material*. School of Computing, Grand Valley State University.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Huang, Z. (1998). Extensions to the k-means algorithm for clustering large data sets with categorical values. *Data Mining and Knowledge Discovery*, *2*(3), 283–304. https://doi.org/10.1023/A:1009769707641

Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, *2*(1), 193–218. https://doi.org/10.1007/BF01908075

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning with applications in Python*. Springer. https://doi.org/10.1007/978-3-031-38747-0

Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, *374*(2065), 20150202. https://doi.org/10.1098/rsta.2015.0202

Kaufman, L., & Rousseeuw, P. J. (1990). *Finding groups in data: An introduction to cluster analysis*. Wiley. https://doi.org/10.1002/9780470316801

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, *28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281–297). University of California Press.

McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform manifold approximation and projection for dimension reduction* (arXiv:1802.03426). arXiv. https://doi.org/10.48550/arXiv.1802.03426

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *2*(11), 559–572. https://doi.org/10.1080/14786440109462720

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, *20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Schubert, E., Sander, J., Ester, M., Kriegel, H.-P., & Xu, X. (2017). DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems*, *42*(3), Article 19. https://doi.org/10.1145/3068335

Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, *6*(2), 461–464. https://doi.org/10.1214/aos/1176344136

Seymour, C. W., Kennedy, J. N., Wang, S., Chang, C.-C. H., Elliott, C. F., Xu, Z., et al. (2019). Derivation, validation, and potential treatment implications of novel clinical phenotypes for sepsis. *JAMA*, *321*(20), 2003–2017. https://doi.org/10.1001/jama.2019.5791

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. In *Proceedings of SPIE 1905, Biomedical Image Processing and Biomedical Visualization* (pp. 861–870). https://doi.org/10.1117/12.148698

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9*, 2579–2605.

Ward, J. H., Jr. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, *58*(301), 236–244. https://doi.org/10.1080/01621459.1963.10500845

Wattenberg, M., Viégas, F., & Johnson, I. (2016). How to use t-SNE effectively. *Distill*. https://doi.org/10.23915/distill.00002